

1주차 과제 발표

애니메이션 1실 김동근

목차

- 주제 및 목표
- 1주차 진행 상황
- TODO 리스트

주제 및 목표



Tom Clancy's The Division2

출처:<https://gigglehd.com/gg/game/6787062>

3인칭 슈팅 게임 데모 개발

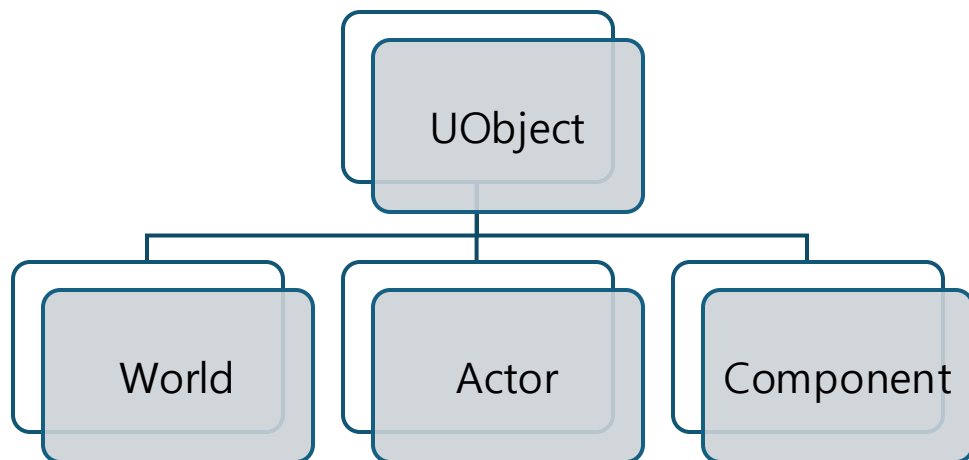
- 3인칭 캐릭터 조작
 - WASD, 마우스 조작
 - 캐릭터 애니메이션
- 사격
 - 투사체 생성
 - 투사체 운동
 - 투사체 소멸
- 피격된 물체의 물리 시뮬레이션
 - 투사체와 상자의 충돌 -> 상자가 움직이고, 넘어짐

1주차 진행 상황

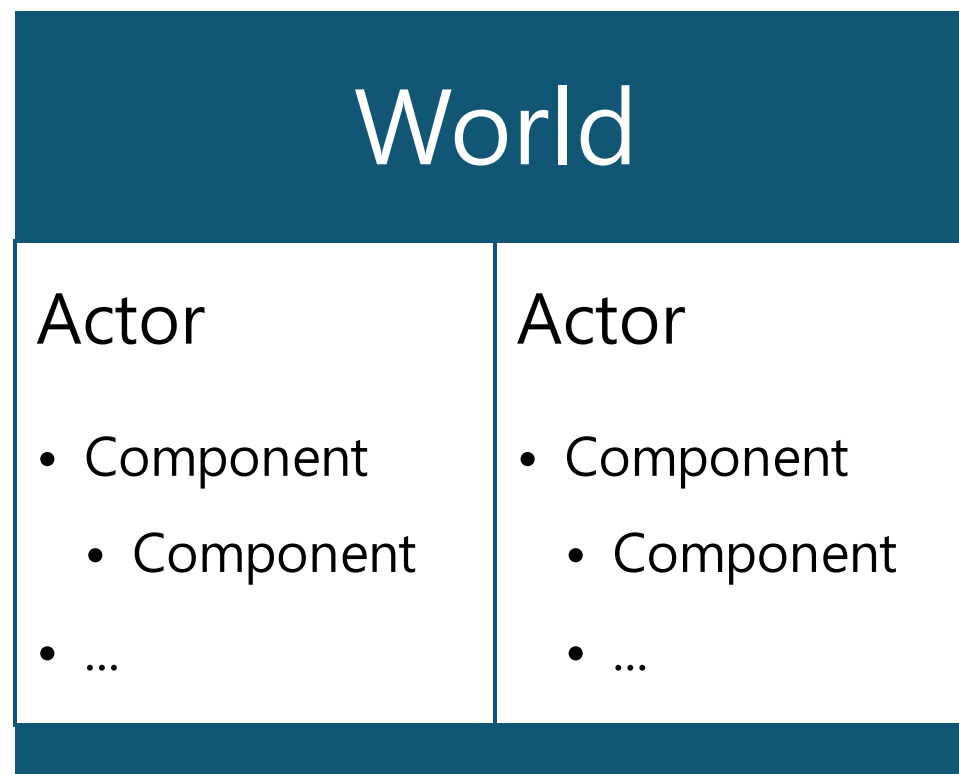
- World-Actor-Component 기반 아키텍처
- Projectile Movement Component

World-Actor-Component 기반 아키텍처

- Unreal Engine의 구조



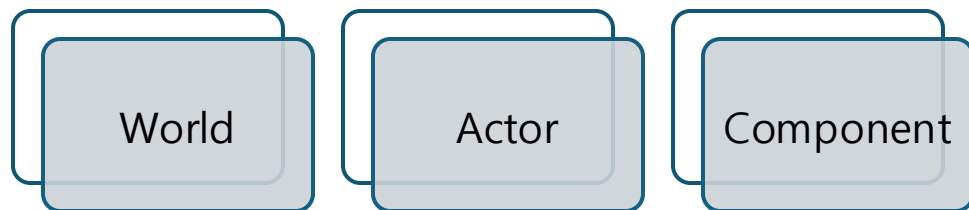
클래스 계층도



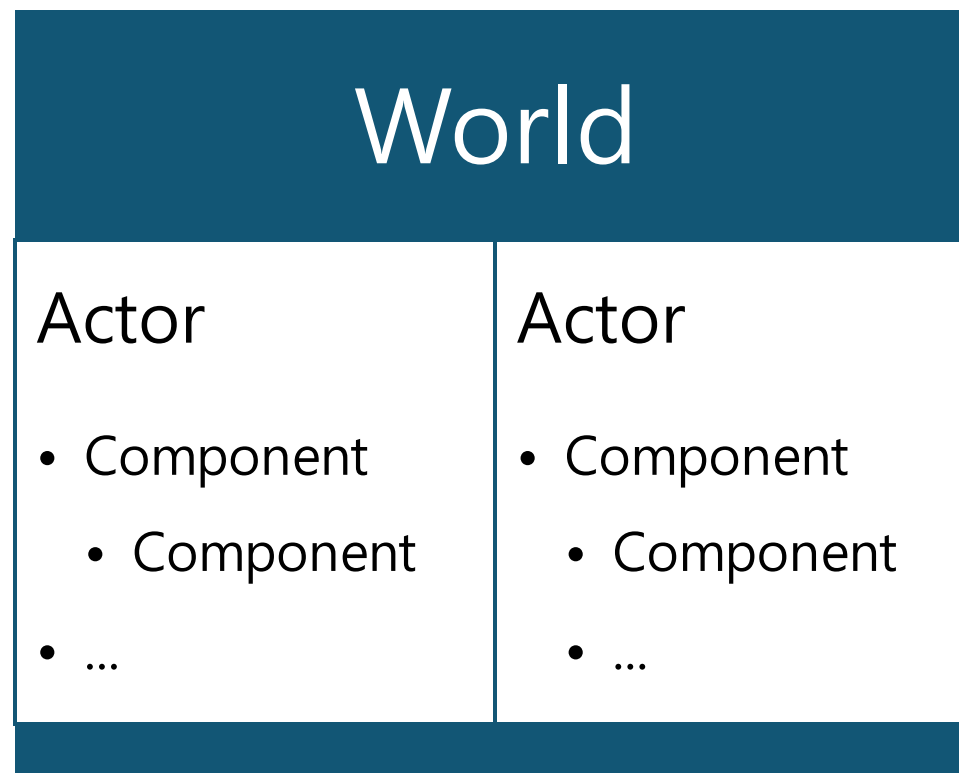
포함 관계

World-Actor-Component 기반 아키텍처

- 구현



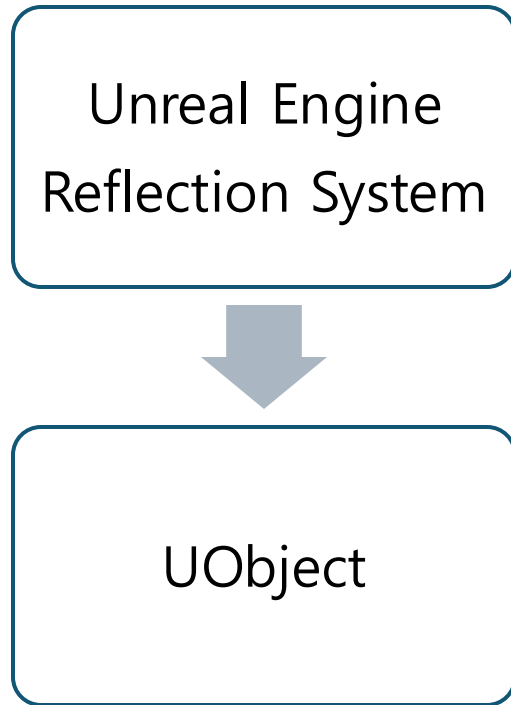
클래스 계층도



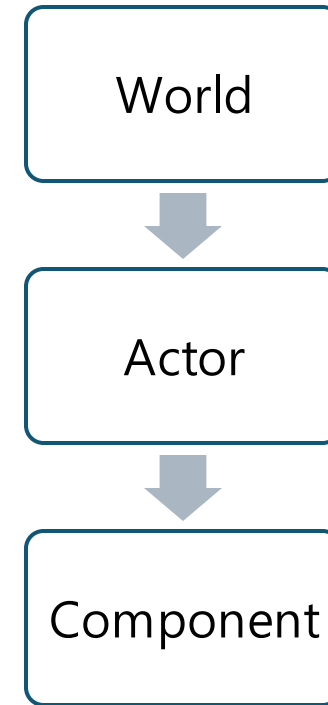
포함 관계

World-Actor-Component 기반 아키텍처

- 객체의 생성과 파괴



언리얼 엔진



실제 구현

World-Actor-Component 기반 아키텍처

```
template<typename T>
inline T* WWorld::SpawnActor()
{
    UINT Index = (UINT)mAllActors.Add(std::make_unique<T>());
    T* Actor = dynamic_cast<T*>(mAllActors[Index].get());
    Actor->SetWorld(this);
    Actor->SetActorId(Index);

    if (Actor->GetRootComponent() == nullptr)
    {
        WSceneComponent* DummyRoot = Actor->CreateComponent<WSceneComponent>();
        Actor->SetRootComponent(DummyRoot);
    }

    Actor->BeginPlay();

    return Actor;
}
```

World-Actor-Component 기반 아키텍처

```
void WWorld::DestroyActor(AActor* Actor)
{
    if (Actor->IsValid())
    {
        Actor->MarkPendingKill();
        DestroyQueue.push_back(Actor);
    }
}
```

DestroyActor 호출시 **DestroyQueue**에 삽입

World-Actor-Component 기반 아키텍처

```
void WWorld::Tick(float Delta)
{
    mRenderItemProxy.Cleanup();
```

```
    for (size_t i = 0; i < mAllActors.Size(); ++i)
    {
        mAllActors[i]->Tick(Delta);
    }

    for (size_t i = 0; i < mAllActors.Size(); ++i)
    {
        auto& Components = mAllActors[i]->GetAllComponents();
        for (size_t j = 0; j < Components.Size(); ++j)
        {
            Components[j]->TickComponent(Delta);
        }
    }
}
```

DestroyActor 호출 예상 타이밍

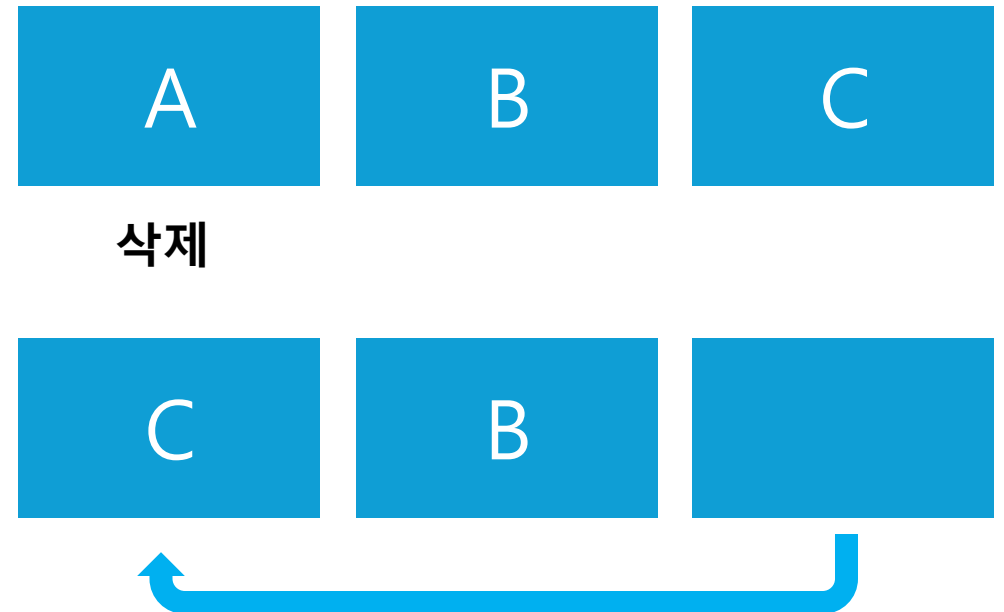
FlushDestroyQueue(); Actor를 파괴하는 함수

World-Actor-Component 기반 아키텍처

```
void WWorld::FlushDestroyQueue()
{
    for (AActor* Actor : DestroyQueue)
    {
        size_t Id = Actor->GetActorId();
        mAllActors.RemoveAt(Id);
        if (Id < mAllActors.Size())
        {
            mAllActors[Id]->SetActorId(Id);
        }
    }

    DestroyQueue.clear();
}
```

RemoveAndSwap 방식으로 액터를 파괴
배열의 요소를 Shift하는 오버헤드 단축
But, 순서를 장담할 수 없음

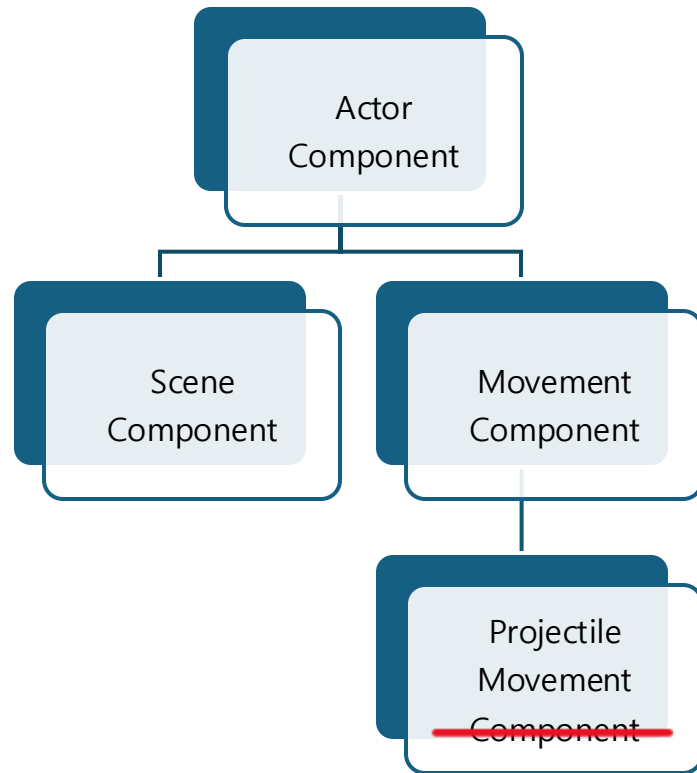


World-Actor-Component 기반 아키텍처

```
WTestWorld::WTestWorld()  
{  
    APawn* Player = SpawnActor<AGhostCameraPawn>();  
    SetPlayer(Player);  
}
```

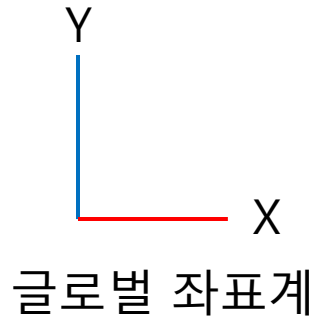
```
AGhostCameraPawn::AGhostCameraPawn()  
{  
    mCameraComponent = CreateComponent<WCameraComponent>();  
    SetRootComponent(mCameraComponent);  
}
```

Projectile Movement Component

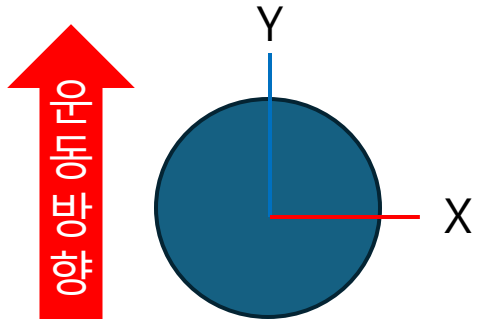


투사체를 Component형태로 구현(UE와 동일)

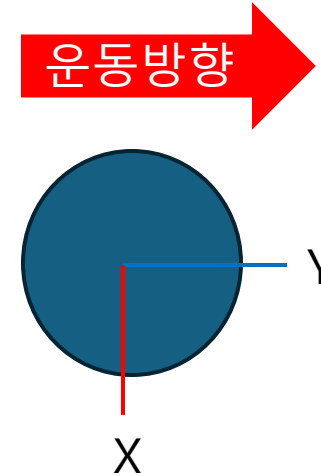
Projectile Movement Component



Velocity = (0, 1, 0)



회전 = (0, 0, 0)



회전 = (0, 0, 90)

Projectile Movement Component

```
void WMovementComponent::TickComponent(float DeltaTime)
{
    Super::TickComponent(DeltaTime);

    // 프레임마다 Owner를 이동시킴
    XMFLOAT4 Quat = GetOwner()->GetActorTransform().GetQuaternionRotation();
    XMVECTOR V = XMVector3Rotate(XMLoadFloat3(&mVelocity), XMLoadFloat4(&Quat));

    XMFLOAT3 ScaledVelocity;
    XMStoreFloat3(&ScaledVelocity, XMVectorScale(V, DeltaTime));
    MoveOwner(ScaledVelocity);
}
```

현재 Velocity와 Owner의 Rotation에 따라
Owner를 운동하도록 구현
(Owner: Component를 소유한 액터)

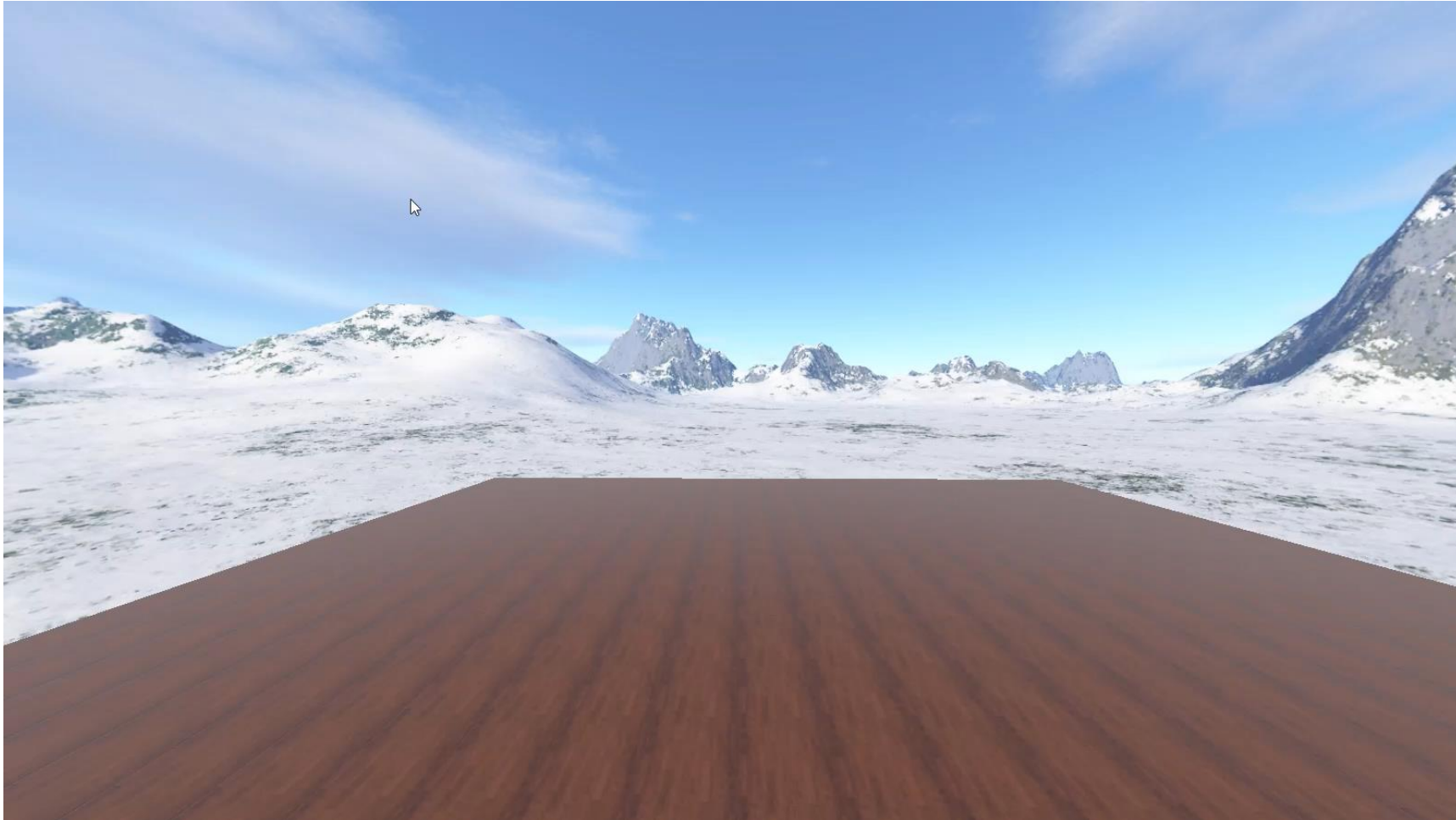
Projectile Movement Component

```
void WProjectileMovementComponent::TickComponent(float DeltaTime)
{
    Super::TickComponent(DeltaTime);

    mLifeTimeElapsed += DeltaTime;
    if (mLifeSpan > 0 && mLifeTimeElapsed > mLifeSpan)
    {
        GetOwner()->Destroy();
    }
}
```

투사체가 설정된 수명 이후에 파괴되는 로직

Projectile Movement Component



TODO 리스트

- Projectile에 가속도, 중력 기능 추가. (+ 호밍)
- fbx 파일 импорт 구현
- 캐릭터 애니메이션 파이프라인 구축
- 물체 물리 시뮬레이션 적용(Jolt Physics 물리엔진 사용 예정)

